Maven Project To Connect To Web Servers Using Selenium Dependency

Maven Project with Multiple Websites Automation Under Chrome Browser

This Maven project uses Selenium and Apache Maven to automate testing of multiple websites using the Chrome browser.

In a single project, Selenium WebDriver with ChromeDriver is used to connect and perform automated actions on three different websites. Maven manages all Selenium dependencies and project build configurations through the pom.xml file.

Websites Included in the Project

1. Google
2. Automation Exercise
3. Practice Test Automation Website

Project Working

1. Maven downloads Selenium and required libraries automatically.
2. ChromeDriver launches the Chrome browser.
3. Selenium connects to all three websites one by one.
4. Automated operations such as navigation, login testing, searching, and validation are performed.
5. Results are verified and displayed in the console.

Features

- Single Maven project for multiple websites
- Browser automation using Chrome
- Automated web testing
- Dependency management using Maven
- Supports CI/CD integration

Applications

- Cross-website automation testing
- Regression testing
- Functional testing
- DevOps automation pipelines

Advantages

- Reduces manual testing effort
- Faster execution of test cases
- Easy project management
- Improves software quality
- Supports integration with tools like Jenkins

This project demonstrates how Selenium and Maven can be combined to automate multiple web servers/websites efficiently using the Chrome browser in a DevOps environment.

Steps to execute the project:

Step1:

Create a maven project on the terminal with the command:

$mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp04 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

The command can be broken down as:

DgroupId=com.example – project's base package (reverse-domain style).

-DartifactId=MyMavenSeleniumApp04 – defines the name of the project (will also be the folder name).

-DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.

-DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use $cd command to go into the directory.

$gedit pom.xml

pom.xml file is created to configure the Maven project. It defines the project information, adds required dependencies like Guava and Commons IO libraries, and includes build plugins to compile the Java code, run unit tests, and package the application into an executable JAR file.

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
```

```
    <scope>test</scope>
   </dependency>

   <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
   <groupId>org.seleniumhq.selenium</groupId>
   <artifactId>selenium-java</artifactId>
   <version>4.29.0</version>
</dependency>
  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
```

```
        <version>3.0.1</version>
        <configuration>
          <archive>
            <manifestEntries>
              <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
          </archive>
        </configuration>
      </plugin>

</plugins>
</build>
</project>
```

Step 3:

inspect the AutomationExercise webpage to find the element IDs required for automation.

Right-click on the webpage and select Inspect. The HTML code will be displayed. Click on the cursor icon (arrow symbol) available on the top-left of the inspect panel, then hover over the username field to identify its ID, which is user-name.

Similarly, check the search button (ID:submit_search). Make a note of these IDs, as they will be used in the App.java file.

Next,

Edit the App.java file

App.java

```
package com.example;


import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;


/**
 * Hello world!
 *
 */
```

```
public class App
{
   public static void main( String[] args )
   {
      WebDriver driver=new ChromeDriver();


      driver.get("https://www.saucedemo.com/");
      driver.manage().window().maximize();
      driver.findElement(By.id("user-name")).sendKeys("standard_user");
      driver.findElement(By.id("password")).sendKeys("secret_sauce");
      driver.findElement(By.id("login-button")).click();



      driver.get("https://practicetestautomation.com/practice-test-login/");
      driver.manage().window().maximize();
      driver.findElement(By.id("username")).sendKeys("student");
      driver.findElement(By.id("password")).sendKeys("Password123");
      driver.findElement(By.id("submit")).click();



      driver.get("https://automationexercise.com/products");
      driver.manage().window().maximize();
      driver.findElement(By.id("search_product")).sendKeys("blue top");
      driver.findElement(By.id("submit_search")).click();
   }
}
```

Execute command $tree

```
vaishnavi@vaishnavi-virtualbox:~/devops21/Chromeexe$ ls
pom.xml  src  target
vaishnavi@vaishnavi-virtualbox:~/devops21/Chromeexe$ tree
.
├── pom.xml
├── src
│   ├── main
│   │   └── java
│   │       └── com
│   │           └── example
│   │               └── App.java
│   └── test
│       └── java
│           └── com
│               └── example
│                   └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           └── App.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenSeleniumApp04-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
                └── AppTest.class

28 directories, 13 files
vaishnavi@vaishnavi-virtualbox:~/devops21/Chromeexe$
```

Step 4:

Build and run the maven application

Execute the command $mvn clean install

```
vaishnavi@vaishnavi-virtualbox:~/devops21/Chromeexe$ mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.example:MyMavenSeleniumApp04 >------------------
[INFO] Building MyMavenSeleniumApp04 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenSeleniumApp04 ---
[INFO] Deleting /home/vaishnavi/devops21/Chromeexe/target
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenSeleniumApp04 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/Chromeexe/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenSeleniumApp04 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vaishnavi/devops21/Chromeexe/target/classes
[INFO]
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ MyMavenSeleniumApp04 ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/vaishnavi/devops21/Chromeexe/src/test/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenSeleniumApp04 ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/vaishnavi/devops21/Chromeexe/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenSeleniumApp04 ---
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.083 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:3.0.1:jar (default-jar) @ MyMavenSeleniumApp04 ---
[INFO] Building jar: /home/vaishnavi/devops21/Chromeexe/target/MyMavenSeleniumApp04-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ MyMavenSeleniumApp04 ---
[INFO] Installing /home/vaishnavi/devops21/Chromeexe/target/MyMavenSeleniumApp04-1.0-SNAPSHOT.jar to /home/vaishnavi/.m2/repository/com/example/MyMavenSeleniumApp04/1.0-SNAPSHOT/MyMavenSeleniumApp04-1.0-SNAPSHOT
.jar
[INFO] Installing /home/vaishnavi/devops21/Chromeexe/pom.xml to /home/vaishnavi/.m2/repository/com/example/MyMavenSeleniumApp04/1.0-SNAPSHOT/MyMavenSeleniumApp04-1.0-SNAPSHOT.pom
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.032 s
[INFO] Finished at: 2026-03-25T13:12:41+05:30
[INFO] ------------------------------------------------------------------------
vaishnavi@vaishnavi-virtualbox:~/devops21/Chromeexe$
```

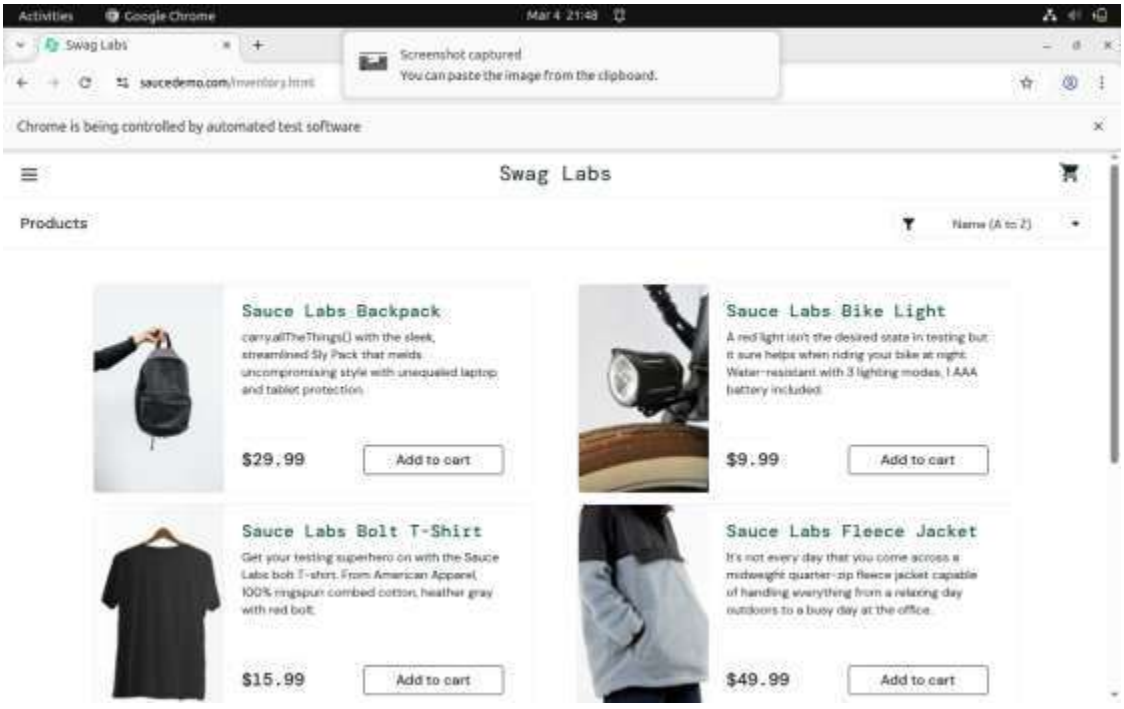
If the command shoes output as “build success”, it means that the project has compiled successfully

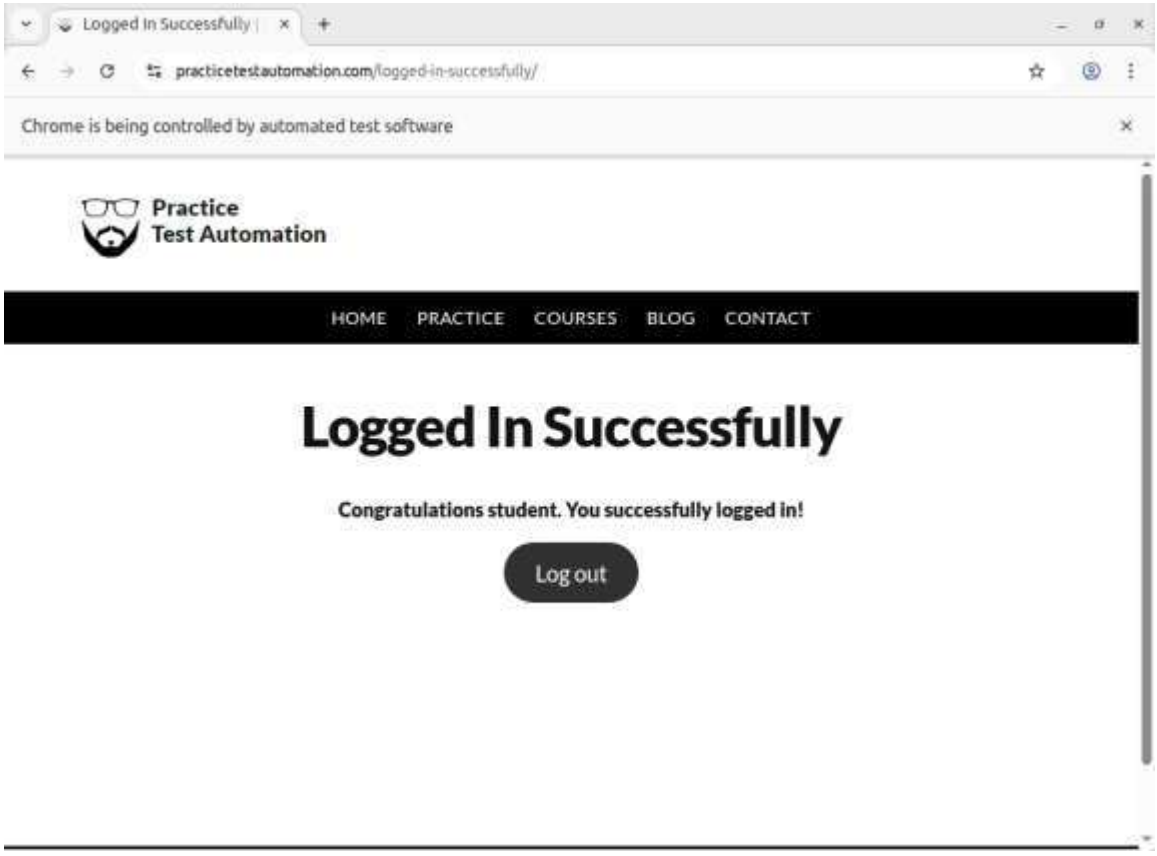
Step 5:

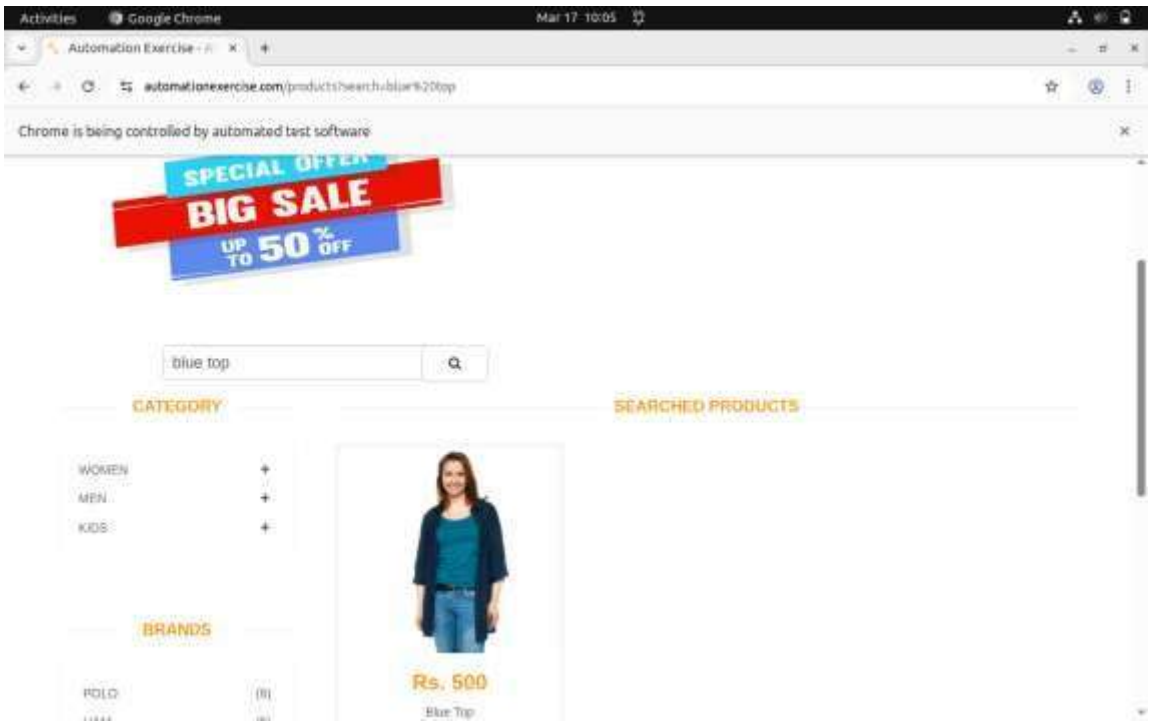
Execute $ java -jar target/MyMavenSeleniumApp03-1.0-SNAPSHOT

If we get error in executing the above jar file do run the below command

$mvn exec:java -Dexec.mainClass="com.example.App"




Logged In Successfully
practicetestautomation.com/logged-in-successfully/
Chrome is being controlled by automated test software
Practice
Test Automation
HOME PRACTICE COURSES BLOG CONTACT
Logged In Successfully
Congratulations student. You successfully logged in!
Log out


Activities
Google Chrome
Mar 17 10:05
Automation Exercise
automationexercise.com/products?search=blue%20top
Chrome is being controlled by automated test software
SPECIAL OFFER
BIG SALE
UP TO 50 % OFF
blue top
CATEGORY
SEARCHED PRODUCTS
WOMEN
MEN
KIDS
BRANDS
POLO
Rs. 500
Blue Top

Sauce Demo Website

Sauce Demo (https://www.saucedemo.com/) is a fake e-commerce website created by Sauce Labs for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Demonstrating cloud testing features
- Learning and practicing test automation

Practice Test Automation Website

https://practicetestautomation.com/practice-test-login/ is a sample website used for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Practicing login functionality testing
- Learning and improving test automation skills

Automation Exercise Website

https://automationexercise.com/products is a sample website used for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Practicing login functionality testing
- Learning and improving test automation skills

Pushing The Project into Git:

Execute the commands:

$ git config --global user.name “vaishnavim1121”

$ git config --global user.email “vaishnavim1121@gmail.com”

$git init

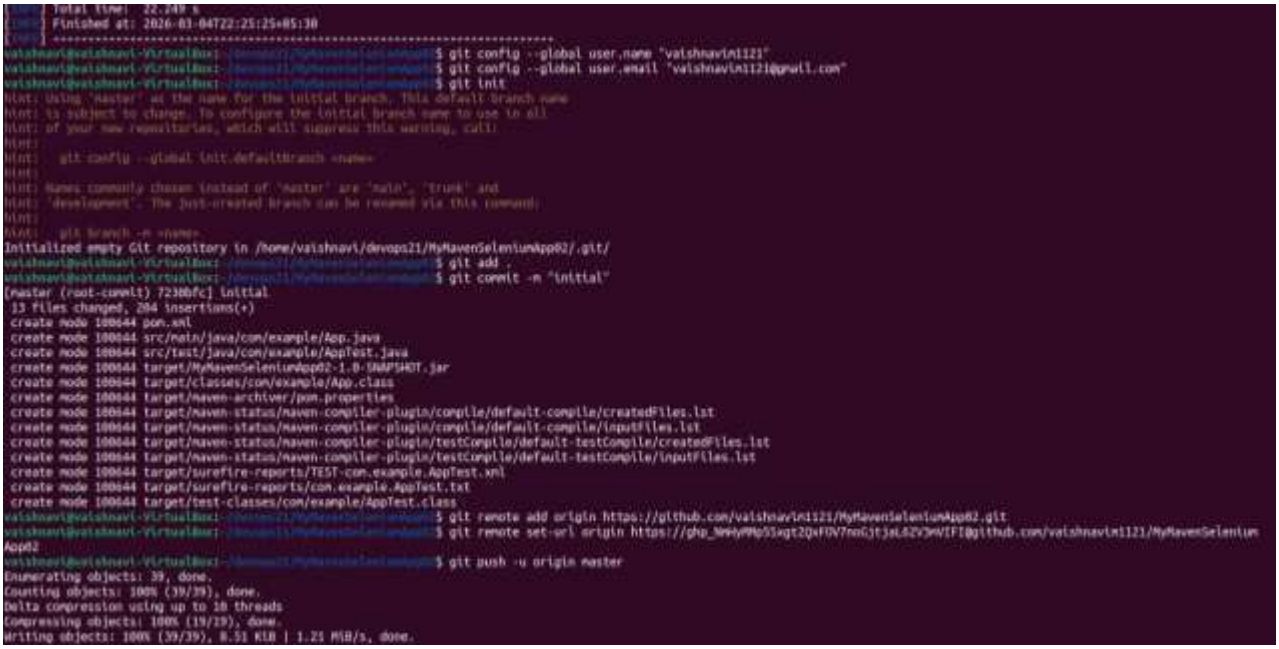
$git add .

$git commit -m “initial commit”

$ git remote add origin https://github.com/vaishnavim1121/MyMavenSeleniumApp.git

$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenSeleniumApp

$ git push -u origin master

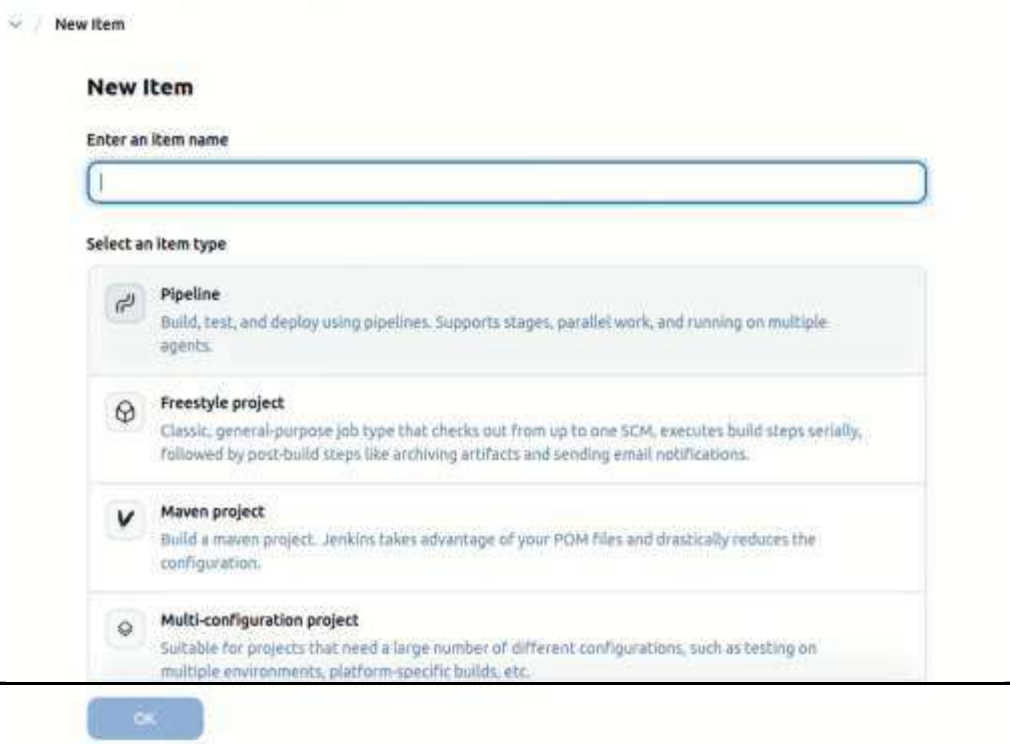


Second Phase

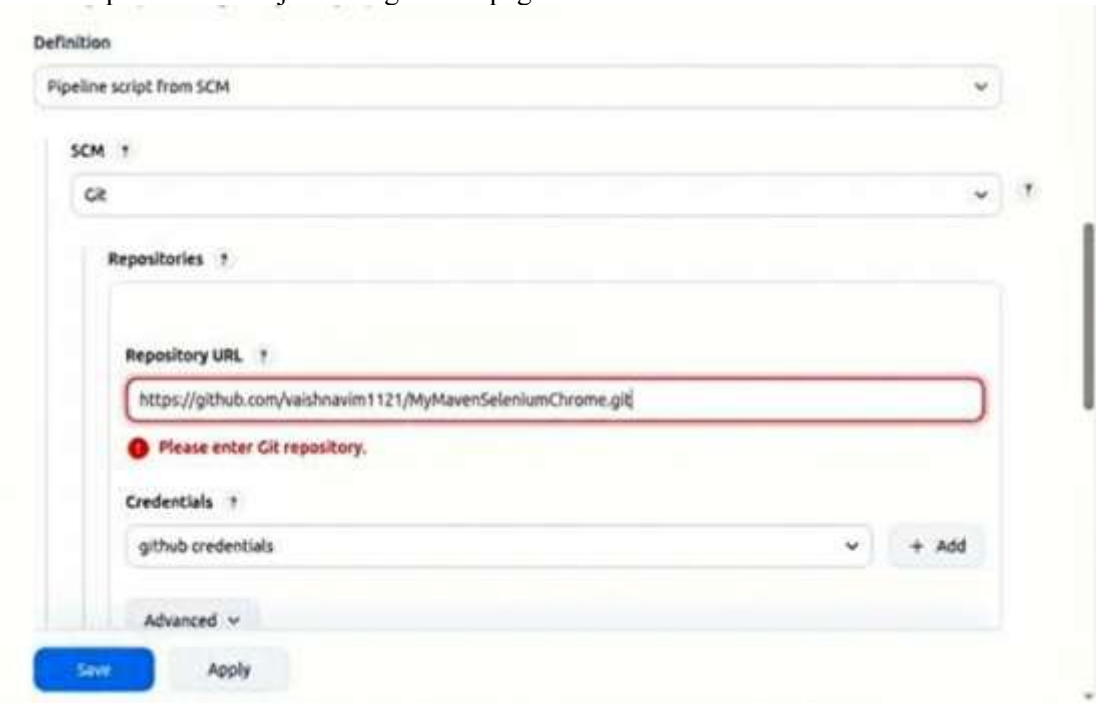
Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository
go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.



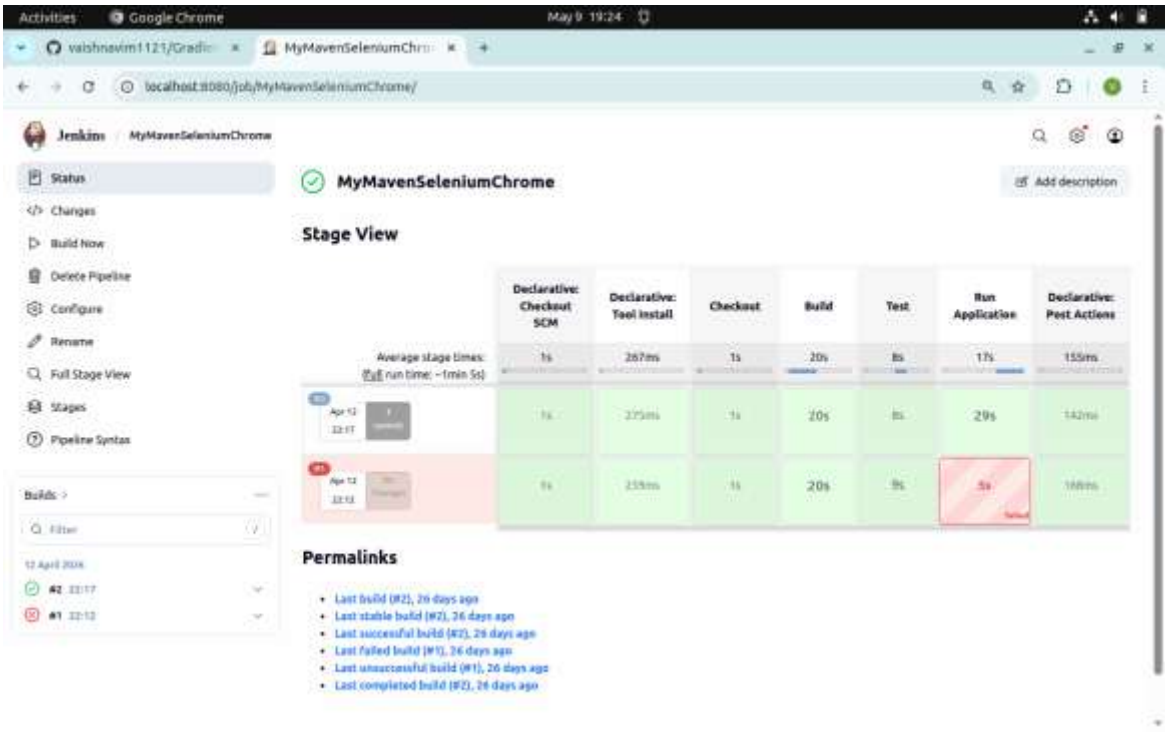
This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click "OK" to proceed to the job configuration page



Now we need to add a jenkinsfile into out project which will consist of the operations that will occur when building the project in Jenkins
It includes step wise instructions guiding at each stage with right commands

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/vaishnavim1121/MyMavenSeleniumApp01.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }



        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar'
            }
```

Now try building the project with Jenkins



The error was due to failure in github repository, by directing the correct link to the Jenkins,and re-building it, the project is built successfully,